

EIP-8182 · DRAFT

EIP-8182: PRIVATE TRANSFERS FOR ETHEREUM

A canonical validity layer for private ETH and compatible ERC-20 transfers

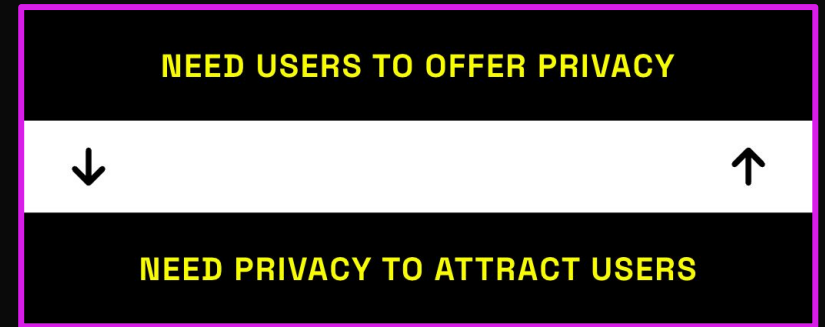
All Core Devs — Execution

May 21, 2026 · Tom Lehman

01 ETHEREUM NEEDS A PRIVATE TRANSFER STANDARD

- Ethereum settles trillions per year in stablecoin transfers alone. Almost every transaction is public.
- No other financial system works this way.
- April 2025: Vitalik called for default-on private sends in existing wallets.
- A year later: still no standard. Fewer than 1 in 10,000 Ethereum transactions in 2025 were private

02 THE APP LAYER WON'T SOLVE THIS ALONE



- **Chicken-and-egg on anonymity sets.** Privacy comes from pool size. A new app starts at zero, so users don't join, so the pool doesn't grow. The biggest pools tend to stay biggest even when a competitor's product is better — and every new pool means weaker privacy for everyone.
- **Trust-model mismatch.** Privacy systems must evolve as bugs are found and cryptography changes. App-layer upgrade authority sits with admins, multisigs, or governance tokens. Public ETH transfers have no such party — private transfers shouldn't either.
- **The better split.** The pool and its anonymity set are public infrastructure. Wallet UX, compliance, proving, relaying, and auth methods live at the app layer

03 WHAT EIP-8182 ACTUALLY IS

- **One system contract.** Installed at 0x00...0081820 at fork activation. No proxy, no admin; changes require a hard fork.
- **UTXO-based shielded pool.** UTXO notes, nullifiers, private in-pool transfers, partially-private deposits and withdrawals.
- **One pool proof verifier.** Groth16 over BN254, Poseidon2, 19 public inputs — embedded in the contract bytecode.
- **Per-user auth proof verifiers.** Each user registers an auth verifier contract of their choosing; pool calls it on every transfer. Protocol doesn't pin an auth method.

How a private transfer works

A simple EIP-8182 transaction flow

01

Sign intent

User signs an EIP-712 intent on a hardware wallet or passkey



02

Send to computer

The signed intent is sent back to the user's computer / wallet app



03

Build proofs

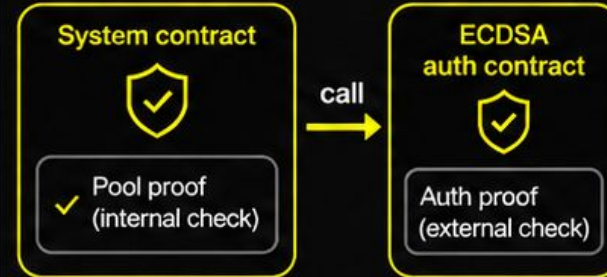
Wallet / prover builds the pool proof and auth proof



04

Verify in transact()

- ✓ System contract verifies the pool proof internally
- ✓ System contract calls an auth verifier contract (example: ECDSA auth contract) to verify the auth proof



05

Update notes

If both checks pass, old notes are consumed and new notes are created

Value moves privately inside the pool



Basic idea: the user signs on secure hardware, the computer builds proofs, and the system contract updates private notes only after both verifications succeed.

04 WHAT EIP-8182 GIVES YOU

- **Sub-second proofs.** 0.15s on an Apple M5 with rapidsnark — mobile-prover viable.
- **Credential / proof separation.** A hardware wallet signs the intent; a phone, desktop, or delegated prover generates the proof. The prover can compute but cannot redirect funds.
- **Synchronous L1 composability.** Unshield → swap → reshield atomically.
- **Minimal core.** Note delivery and encryption are left to companion standards.

05 RISKS, AND HOW THEY'RE HANDLED

- **Groth16/BN254 isn't post-quantum.** Mitigation: The verifier bytecode is swappable in a hard fork, with no node migration or user action required
- **State growth.** Each transfer adds unprunable nullifier state. Deliberate: nullifiers live in EVM state, preserving synchronous composability and tooling compatibility, and it's buildable today with no other protocol changes. The set can migrate to a more scalable state type in a future fork if scale demands.
- **Pool circuit / verifier bug potential.** One 42,959-constraint in-protocol circuit. A bug's blast radius is bounded to pool participants — not all of Ethereum.
- **Trusted setup requirement.**
- **Transfer gas cost: 1.69M.** Mostly Poseidon2 hashing. Could mitigate with a precompile.

LET'S FINISH THE JOB

- Native privacy is Ethereum's big missing feature, and one of the few “demand side” features on the Straw Map.
- We're already building much of what enables it — Frames, FOCIL, keyed nonces.
- **The ask:** inclusion in the earliest possible fork.